Código: 750023C Curso: Infraestructuras Paralelas y Distribuidas Periodo: 2026-2
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

TALLER 2: DESCOMPOSICIÓN DE PROBLEMAS EN PYTHON

Objetivo: Este taller tiene como objetivo reforzar la comprensión de los conceptos de descomposición por dominio (datos) y descomposición funcional (tareas) a través de la implementación de dos programas en Python.

Requisitos:

- Conocimientos básicos de Python (funciones, listas, bucles).
- Conocimientos básicos de programación concurrente (deseable, pero no indispensable).

Ejercicios:

Ejercicio 1: Descomposición por Dominio (Datos) - Procesamiento Paralelo de Imágenes

Problema: Tienes una lista grande de rutas a imágenes en un directorio. Necesitas aplicar una función de procesamiento (por ejemplo, convertir a escala de grises) a cada imagen. Implementa una solución que utilice la descomposición por dominio para procesar las imágenes en paralelo, acelerando el proceso.

Paso 1: Preparación

1. **Crear un directorio de imágenes:** Crea un directorio con algunas imágenes de prueba (pueden ser imágenes pequeñas o copias de la misma imagen). Si no tienes imágenes, puedes crear archivos vacíos con extensiones de imagen (ej: imagen1.jpg).
2. **Instalar Pillow (PIL):** Si no lo tienes, instala la biblioteca Pillow (PIL) para el manejo de imágenes (en anaconda online no es necesario): pip install Pillow

Paso 2: Implementación (Descomposición por Dominio)

Cree su algoritmo en python.

Paso 3: Implementación (Secuencial para comparar)



Código: 750023C Curso: Infraestructuras Paralelas y Distribuidas Periodo: 2026-2
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

```
from PIL import Image
import os
import time

def convertir_a_gris(ruta_imagen):
    """Convierte una imagen a escala de grises."""
    try:
        imagen = Image.open(ruta_imagen)
        imagen_gris = imagen.convert('L')  # 'L' representa escala de grises
        nombre_archivo, extension = os.path.splitext(ruta_imagen)
        ruta_gris = nombre_archivo + "_gris" + extension
        imagen_gris.save(ruta_gris)
        print(f"Imagen convertida: {ruta_imagen} -> {ruta_gris}")
    except FileNotFoundError:
        print(f"Error: No se encontró la imagen {ruta_imagen}")
    except Exception as e:
        print(f"Error al procesar {ruta_imagen}: {e}")

def procesar_imagenes_secuencial(lista_imagenes):
    """Procesa una lista de imágenes secuencialmente."""
    for ruta_imagen in lista_imagenes:
        convertir_a_gris(ruta_imagen)

if __name__ == '__main__':
    directorio_imagenes = "imagenes_prueba"  # Reemplaza con el nombre de tu
directorio
    lista_imagenes = [os.path.join(directorio_imagenes, f) for f in
os.listdir(directorio_imagenes) if
os.path.isfile(os.path.join(directorio_imagenes, f))]

    inicio = time.time()
    procesar_imagenes_secuencial(lista_imagenes)
    fin = time.time()

    print(f"Tiempo total de procesamiento secuencial: {fin - inicio:.2f}
segundos")
```

Código: 750023C Curso: Infraestructuras Paralelas y Distribuidas Periodo: 2026-2
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Paso 4: Prueba y Comparación

1. Ejecuta ambos programas (el paralelo y el secuencial).
2. Compara los tiempos de ejecución. Deberías observar que la versión paralela es más rápida, especialmente si tienes múltiples núcleos en tu CPU y una cantidad significativa de imágenes.

Ejercicio 2: Descomposición Funcional (Tareas) - Pipeline de Procesamiento de Texto

Problema: Necesitas procesar un archivo de texto grande aplicando una serie de transformaciones: leer, dividir en líneas, limpiar cada línea (eliminar espacios en blanco al principio y al final), convertir a mayúsculas y escribir el resultado en un nuevo archivo. Implementa este proceso utilizando una descomposición funcional, donde cada transformación sea una tarea separada.

Paso 1: Preparación

1. **Crear un archivo de texto de prueba:** Crea un archivo de texto llamado texto_entrada.txt con varias líneas de texto (algunas con espacios en blanco al principio y al final, algunas en minúsculas).

Ejemplo de texto_entrada.txt:

text
    esto es una linea con espacios.
otra linea en minusculas
      Y OTRA CON ESPACIOS Y MAYUSCULAS

Paso 2: Implementación (Descomposición Funcional)

Cree su algoritmo en python

Paso 3: Implementación (Secuencial para comparar)

Código: 750023C Curso: Infraestructuras Paralelas y Distribuidas Periodo: 2026-2
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

```
import time

def procesar_texto_secuencial(ruta_entrada, ruta_salida):
    """Procesa el archivo de texto secuencialmente."""
    try:
        with open(ruta_entrada, 'r') as f_in, open(ruta_salida, 'w') as
f_out:
            for linea in f_in:
                linea_limpia = linea.strip()
                linea_mayusculas = linea_limpia.upper()
                f_out.write(linea_mayusculas + '\n')
    except FileNotFoundError:
        print(f"Error: No se encontró el archivo {ruta_entrada}")

if __name__ == '__main__':
    ruta_entrada = "texto_entrada.txt"
    ruta_salida = "texto_salida_secuencial.txt"

    inicio = time.time()
    procesar_texto_secuencial(ruta_entrada, ruta_salida)
    fin = time.time()

    print(f"Tiempo total de procesamiento secuencial: {fin - inicio:.2f}
segundos")
    print(f"Archivo procesado secuencialmente guardado en {ruta_salida}")
```

Paso 4: Prueba y Comparación

1. Ejecuta ambos programas (el pipeline y el secuencial).
2. Compara los tiempos de ejecución y el contenido de los archivos de salida. Deberías observar que el archivo texto_salida.txt contiene el mismo contenido que texto_salida_secuencial.txt, pero que el pipeline (con descomposición funcional) puede ser más rápido, especialmente para archivos grandes.

Generar una Explicación Detallada de las Soluciones y conclusiones que obtienen del ejercicio en un documento formal.